Correspondances

Votre attention s'il vous plaît. Le TER numéro 8756 à destination de Marseille-Saint-Charles entre en gare, voie F. Éloignez-vous de la bordure du quai.

Introduction

Une région confie à votre équipe la refonte du système de billetterie de son réseau ferroviaire. Le réseau est modeste : une trentaine de gares, une douzaine de lignes, des trains qui circulent selon des jours de la semaine différents. Mais la billetterie, elle, doit encaisser plusieurs centaines de milliers de réservations par an sans faiblir.

Ce projet n'est pas une découverte de PostgreSQL. Vous en avez déjà écrit en septembre, et vous savez faire un CRUD sur trois tables. Cette semaine est celle du passage à l'échelle professionnelle, sur trois plans :

- la modélisation formelle, parce qu'un domaine ferroviaire ne se devine pas et que trois entités n'y suffiront pas ;
- la migration, parce qu'en production on ne recrée jamais une base qui contient déjà des données ;
- la performance, parce qu'une requête qui répond en 4 millisecondes sur 100 lignes peut mettre 12 secondes sur 500 000.

Il n'y a presque pas d'interface à écrire cette semaine. Le livrable est la base, sa conception, son évolution et ses performances.

Étape 1 : le domaine à modéliser

Le cahier des charges du client, à traduire en modèle.

Le réseau

Le réseau est composé de gares. Une ligne relie plusieurs gares dans un ordre donné. Une même gare peut appartenir à plusieurs lignes, et c'est ce qui rend les correspondances possibles.

Les circulations

Un train circule sur une ligne, selon un horaire qui indique, pour chaque gare desservie, une heure d'arrivée et une heure de départ. Tous les trains ne desservent pas toutes les gares de leur ligne : certains sont directs.

Un train ne circule pas tous les jours. Chaque circulation obéit à un calendrier : tous les jours, en semaine seulement, week-end seulement, ou jours exceptionnels.

Le matériel

Un train est composé de voitures, chaque voiture contient des places numérotées, chaque place appartient à une classe de confort.

Les tarifs

Le prix d'un trajet dépend de la distance parcourue, de la classe, d'une période tarifaire (heure creuse, heure de pointe, période de vacances) et d'un profil voyageur (plein tarif, jeune, senior, abonné).

Les réservations

Un voyageur réserve une place, pour une date de circulation donnée, entre une gare de départ et une gare d'arrivée. Attention au point qui piège tout le monde : une place se réserve pour un segment de trajet, pas pour un train

entier. Si un voyageur descend à mi-parcours, la place doit pouvoir être revendue pour la suite du trajet.

Les points durs, assumés

Ils sont dans le sujet parce qu'ils sont formateurs, pas par méchanceté :

- une desserte est une association porteuse de données entre un train et une gare, avec des horaires. Ce n'est pas une simple clé étrangère ;
- un tarif dépend de plusieurs dimensions à la fois ;
- une place réservée sur un segment oblige à réfléchir avant de coder.

Ce qui est attendu

Le cycle complet : **MCD, puis MLD, puis MPD**. Chacun sous forme d'image plus une explication textuelle des choix.

Ces trois documents sont commités et poussés avant la première entité JPA du projet. L'ordre des commits est vérifié à l'évaluation. Une relecture par l'accompagnateur est possible et recommandée, mais elle ne bloque pas votre avancement : le sujet n'attend personne.

Rappel : **le code s'écrit en anglais**, donc les tables, colonnes, contraintes et entités aussi.

Étape 2 : la base et ses contraintes

Vous créez le schéma PostgreSQL 16 correspondant à votre MPD.

Les contraintes ne sont pas décoratives, elles sont évaluées :

- clés primaires et clés étrangères, avec un comportement à la suppression choisi et justifié ;
- contraintes `UNIQUE` composites là où le métier l'impose. Une place ne peut pas être réservée deux fois sur le même segment et la même date
- contraintes `CHECK` sur les valeurs impossibles : une heure d'arrivée après l'heure de départ suivante, un prix négatif, un numéro de voiture nul
- valeurs par défaut et NOT NULL posés sciemment.

Le principe à retenir et à savoir défendre : **une règle métier tenue par la base vaut mieux qu'un `if` dans un service**. Un if se contourne par un accès direct en SQL, par un script d'import, ou par une seconde instance de l'application. Une contrainte, non.

Étape 3 : le générateur de volumétrie

Sans volume, aucune mesure n'a de sens.

Le générateur s'écrit en Java ou en SQL. Le SQL est parfaitement adapté ici, et generate_series fait partie de ce que vous devez savoir manipuler. Ce n'est ni un script Python ni un script shell : le périmètre de langages du B2 s'applique.

Exigences :

- **au moins 500 000 réservations**

- réparties sur une année de circulation, avec des jours plus chargés que d'autres
- des données cohérentes : une réservation pointe vers une circulation qui existe réellement ce jour-là.

Le générateur est un livrable. Il doit se relancer proprement, documenté dans le README

Étape 4 : la couche d'accès

Vous branchez JPA et Hibernate sur ce schéma, avec Spring Data.

Périmètre volontairement réduit : quelques endpoints de lecture suffisent à prouver que la couche fonctionne. Il n'y a pas d'interface à produire.

- Entités JPA fidèles au MPD, relations correctement annotées.
- Requêtes dérivées ou JPQL paramétré. Aucun SQL concaténé à la main, jamais.

Étape 5 : l'évolution du cahier des charges

Annexe à traiter une fois l'étape 4 terminée, le schéma initial tagué `v1` et la base remplie. Ne l'anticipez pas : si vous intégrez cette évolution dès le lundi dans votre modèle, vous n'aurez aucune migration à montrer, donc rien à livrer sur le point central de la semaine.

« Demande du client La région veut pouvoir consulter l'historique des prix. Aujourd'hui, quand un tarif change, l'ancien disparaît. Nous voulons désormais conserver tous les tarifs passés, avec leur période de validité, et

pouvoir répondre à la question : combien coûtait ce trajet le 3 mars dernier ? Bien entendu, les réservations existantes doivent rester intactes et continuer à pointer vers le prix qui leur a été appliqué au moment de l'achat. »

Ce que cela implique :

- une migration versionnée avec Flyway ou Liquibase ;
- une migration qui transforme les données existantes, pas seulement la structure
- une base qui se migre et ne se recrée pas. Supprimer et régénérer est un échec sur cette étape, même si le résultat final est correct.

Étape 6 : performance

Le dossier de mesure est un livrable à part entière.

Analyse de plan

Au moins une requête analysée avec EXPLAIN ANALYZE, avant et après optimisation. Vous devez savoir lire et commenter la différence entre un parcours séquentiel et un parcours d'index, et entre l'estimation du planificateur et la réalité mesurée.

Index

Au moins un index créé, justifié, avec sa mesure avant et après. Si l'index est composé, l'ordre des colonnes doit être expliqué.

Le N+1

Vous devez identifier un problème de N+1 dans votre propre code JPA, puis le corriger avec JOIN FETCH, @EntityGraph ou une projection.

La méthode : activez le log SQL d'Hibernate, appelez un endpoint de liste, et comptez les requêtes réellement exécutées. La preuve avant et après est jointe au dossier.

Requêtes SQL

Cinq requêtes non triviales, commentées, mêlant jointures multiples, agrégations, sous-requêtes ou fonctions fenêtre. Le classement des lignes les plus réservées se prête particulièrement bien à une fonction fenêtre.

Livrables

La correspondance, enfin calculée.

Vous avez modélisé tout un réseau, et vous ne vous en êtes jamais servis pour trouver un trajet. Écrivez une requête qui, entre deux gares que ne relie aucune ligne commune, propose un itinéraire avec au plus une correspondance, en respectant un temps de battement minimum à l'échange. Une CTE récursive est l'outil naturel. C'est la requête qui donne enfin du sens à la modélisation de la semaine.

La migration qui doit d'abord nettoyer.

Ajoutez une contrainte que vos cinq cent mille lignes existantes violent déjà, par exemple une unicité que vous découvrez trompeuse après coup. Un simple `ALTER TABLE` échouera : la migration doit détecter les lignes fautives, décider quoi en faire, tracer sa décision, puis poser la contrainte, le tout rejouable. C'est le scénario de migration le plus fréquent en production, et le seul qui fait vraiment peur.

Le partitionnement.

Découpez la table des réservations par mois ou par année avec le partitionnement natif de PostgreSQL 16, puis remesurez une requête filtrée sur une période. Vérifiez dans le plan d'exécution que les partitions inutiles sont réellement écartées. Regardez aussi ce que le partitionnement coûte à une requête qui, elle, traverse toute l'année : un gain quelque part se paie presque toujours ailleurs

Livrables

---

1. MCD, MLD et MPD, en image plus texte, poussés avant la première entité JPA.
2. Schéma PostgreSQL avec ses contraintes, sous forme de scripts versionnés.
3. Générateur de volumétrie, en Java ou en SQL, relançable.
4. Couche JPA fonctionnelle avec quelques endpoints de lecture.
5. Migration versionnée répondant à l'évolution du cahier des charges, rejouable sur une base déjà remplie.
6. Dossier performance : plans d'exécution avant et après, index justifiés, N+1 identifié et corrigé avec preuves.
7. Cinq requêtes SQL commentées.
8. README avec instructions de lancement.

Compétences visées

- BC02-C3 : concevoir et mettre en place une base de données relationnelle, niveau Analyser
- BC02-C4 : développer des composants d'accès aux données SQL, niveau Analyser
- BC02-C2 : définir l'architecture logicielle d'une application

Rendu

Le projet est à rendre sur
https://github.com/prenom-nom/Correspondances
Pensez à donner les droits sur le répertoire à **deepthoughtlaplateforme** !

Structure attendue :
Correspondances/
src/ (projet Spring Boot, entités et migrations)
db/ (scripts de génération de volumétrie, requêtes commentées)
docs/ (MCD, MLD, MPD, dossier performance)
README.md

Base de connaissances

PostgreSQL

- Documentation officielle 16
- Contraintes
- Index
- Utiliser EXPLAIN
- Fonctions fenêtre
- Fonctions de génération de séries

Modélisation

- Merise, MCD MLD MPD, cours de référence en accès libre

JPA et Hibernate

- Spring Data JPA
- Hibernate, guide utilisateur
- Entity graphs

Migrations

- Flyway, documentation
- Liquibase, documentation